Triage de alertas SOC sobre 2M de eventos con modelos tabulares, de texto y de grafo: propuesta de proyecto

Author Name
Curso: Aprendizaje Automático — Semestre 8

***Resumen*—Los centros de operaciones de seguridad se enfrentan a un volumen de alertas que ningún equipo humano puede revisar: nuestra captura contiene 2,011,674 eventos en seis días con apenas 0.38 % de eventos maliciosos. Proponemos investigar el *triage de alertas* como problema de clasificación multiclase con desbalance extremo y deriva temporal. El estudio revisa dos líneas de literatura verificadas (Devign, aprendizaje sobre grafos de código; CVSS-BERT, texto a severidad), formula la tarea de aprendizaje, especifica cinco metodologías de preprocesamiento y tres modelos con su análisis de complejidad, y fija el protocolo de evaluación. Este documento presenta el diseño del estudio; los resultados experimentales se reportarán en la entrega final.**

***Index Terms*—triage de alertas, desbalance de clases, SOC, grafo de proveniencia, MITRE ATT&CK, detección de intrusiones.**

I. Introducción

I-A. Contexto

Un centro de operaciones de seguridad (SOC) vigila la infraestructura de una organización mediante múltiples productos heterogéneos: firewalls y pasarelas, protección de endpoints y EDR, directorios de identidad, plataformas de nube, hipervisores y servicios de red. Cada producto emite registros de forma continua (syslog, auditoría de Windows, flujos de red, telemetría de endpoint, eventos de autenticación). Una plataforma de correlación —en nuestro caso WitFoo Precinct versión 6.x— normaliza esos registros y aplica un catálogo de 261 reglas de detección; cuando varias señales relacionadas se agrupan, la plataforma levanta un *incidente*, compuesto por las señales disparadoras, un subgrafo de proveniencia que describe qué entidad tocó a qué entidad, y un reporte de caza en lenguaje natural.

El conjunto de datos empleado, `witfoo/precinct6-cybersecurity` versión 2.1.0, contiene 2,011,674 eventos capturados en producción entre el 26 de julio y el 1 de agosto de 2024, pertenecientes a 4 organizaciones, 19 productos y 12 fabricantes.

I-B. Motivación

Cuatro razones justifican el estudio. Primero, el **volumen**: 2,011,674 eventos en seis días imposibilitan la revisión manual. Segundo, el **desbalance extremo**: solo 7,728 eventos (0.38 %) son leads maliciosos dentro de la ventana, frente a 1,897,153 benignos, lo que da una razón de desbalance aproximada de 245:1. Bajo esa razón la exactitud es una métrica engañosa: un clasificador trivial que siempre responda "benigno" alcanza 94.31 %. Tercero, la **asimetría de costos**: en seguridad el costo de un falso negativo (una brecha no detectada) es órdenes de magnitud mayor que el de un falso positivo, por lo que la optimización debe orientarse al recall de la clase minoritaria y no a la exactitud global. Cuarto, el **cambio de contexto del analista**: decidir si escalar o monitorear requiere el mensaje crudo, el mapeo a MITRE ATT&CK y la cadena del ataque, no un único puntaje.

I-C. Objetivos y pregunta de investigación

El objetivo general es diseñar y evaluar un sistema de triage que, dado un evento de seguridad, estime su nivel de riesgo con probabilidades calibradas y produzca una recomendación operativa. La pregunta de investigación que guía el estudio es:

> *¿Es posible recuperar eventos maliciosos en un flujo de alertas SOC con desbalance 245:1 y deriva temporal combinando representaciones tabulares, textuales y estructurales del evento, y qué compromiso existe entre la recuperación de la clase minoritaria y la del nivel intermedio?*

Los objetivos específicos son: (i) caracterizar el conjunto de datos y verificar la presencia de leakage y deriva temporal; (ii) implementar al menos cinco metodologías de preprocesamiento; (iii) entrenar y comparar tres modelos con análisis de complejidad; (iv) desplegar el sistema como un servicio y una consola de analista.

II. Trabajo previo

II-A. Marco de referencia

La literatura relevante se organiza en tres familias. La primera es la **detección basada en grafos de proveniencia**, que modela la actividad del sistema como un grafo de entidades y eventos: NODLINK [8] aborda detección de APT en línea, KnowHow [9] integra inteligencia de amenazas para reducir falsos positivos, FLASH [10] combina redes neuronales de grafo con semántica temporal, y threaTrace [11] aplica GraphSAGE [12] a nivel de nodo. La segunda es el **mapeo automático a MITRE ATT&CK** desde texto, representada por TRAM [13] y por estudios comparativos de representaciones textuales [14]. La tercera agrupa técnicas para **desbalance de clases**, en particular Focal Loss [15] y formulaciones sensibles al costo donde el costo de un falso negativo domina [16].

Dos trabajos se analizan en detalle por su relación directa con las representaciones que emplearemos: uno opera sobre estructuras de grafo y otro sobre texto.

II-B. Caso 1: Devign — aprendizaje sobre grafos de código

Zhou *et al.* [1] (NeurIPS 2019) proponen Devign, un modelo de red neuronal de grafos para identificación de vulnerabilidades. Su enfoque construye una representación compuesta del código —árbol de sintaxis abstracta como esqueleto, con dependencias de control y de datos codificadas como aristas de distintos tipos— y entrena una red recurrente de grafos con compuertas más un módulo convolucional propio que selecciona jerárquicamente los nodos relevantes para la clasificación a nivel de grafo.

El problema se formula como **clasificación binaria a nivel de función**: determinar si una función dada es vulnerable o no. Los datos provienen de cuatro librerías populares escritas en C y fueron etiquetados manualmente, con un costo reportado de aproximadamente 600 horas-persona, empleando código real y complejo en lugar de ejemplos sintéticos.

El resultado reportado es una mejora promedio de **10.51 % en exactitud y 8.68 % en F1** respecto a los métodos de referencia; adicionalmente supera a analizadores estáticos conocidos por 27.99 % de F1 promedio y alcanza 74.11 % de exactitud sobre funciones correspondientes a 40 CVE recientes.

Para nuestro estudio, Devign es el precedente del componente de grafo: demuestra que una representación estructural permite capturar patrones que no aparecen en atributos aislados. El contraste relevante es el régimen de etiquetado: Devign depende de anotación humana costosa y relativamente limpia, mientras que nosotros dispondremos de etiquetas automáticas, baratas pero ruidosas.

II-C. Caso 2: CVSS-BERT — severidad a partir de texto

Shahid y Debar [2] (IEEE ICMLA 2021) abordan un problema distinto: trabajan con **texto, no con código**. Su motivación es operativa: cuando se publica una vulnerabilidad solo se dispone de una descripción textual, y el análisis por expertos que asigna el vector CVSS puede tardar días, mientras el volumen diario de vulnerabilidades sigue creciendo.

El enfoque entrena **varios clasificadores BERT, uno por cada métrica del vector CVSS** (vector de ataque, complejidad de ataque, privilegios requeridos, interacción de usuario, alcance, e impactos en confidencialidad, integridad y disponibilidad), tomando como entrada la descripción textual de la vulnerabilidad. Los valores predichos por cada clasificador se concatenan para formar el vector, a partir del cual se calcula el puntaje de severidad. Emplean 45,926 descripciones de CVE del período 2018–2020 y reportan exactitudes por métrica entre 83.79 % y 96.07 %, con un error absoluto medio de 0.73 en el puntaje de severidad y coincidencia exacta en 55.3 % de los casos: la severidad calculada queda muy cercana a la asignada por un experto humano. Incluyen además un método de explicabilidad por gradientes que identifica las palabras más influyentes en cada predicción.

La limitación señalada por estudios posteriores es que **los puntajes CVSS varían según quién los asigna**. En particular, se ha documentado que los puntajes pueden divergir hasta 0.8 puntos entre NVD y otros proveedores, y hasta 1.3 puntos según la versión del estándar empleada [3]; que NVD tiende a asignar valores mayores que los CNA y que el desacuerdo aparece con mayor frecuencia en sub-métricas dependientes del producto [4]; y que la inconsistencia entre fuentes puede degradar el desempeño de predicción de severidad aguas abajo hasta en 77 % [5]. En consecuencia, las etiquetas dependen del contexto del asignador.

Para nuestro proyecto, CVSS-BERT cumple tres funciones: es el precedente del componente de texto (descripción a etiqueta), su método de explicabilidad inspira la explicabilidad que expondremos al analista, y su limitación documentada fundamenta directamente nuestra sección de limitaciones, dado que nuestras etiquetas también provienen de un asignador automático y no de un analista humano.

II-D. Síntesis y brecha

La tabla I resume los dos casos frente al estudio propuesto. La brecha que identificamos es la siguiente: los trabajos revisados operan sobre **código fuente** o sobre **descripciones de vulnerabilidades**, pero no sobre **telemetría de operación en tiempo real** con desbalance extremo, deriva temporal y decisión humana en el bucle. Nuestro aporte previsto es estudiar esa combinación y cuantificar el compromiso entre recuperar la clase minoritaria y preservar el nivel intermedio.

Cuadro I
CASOS ANALIZADOS FRENTE AL ESTUDIO PROPUESTO.

	Devign	CVSS-BERT	Propuesto
Entrada	grafo de código	texto de CVE	tabular + texto + subgr
Tarea	binaria (función)	8 métricas → puntaje	3 niveles + ATT&CK
Etiquetas	manuales (600 h)	NVD (experto)	Precinct (automático)
Modelo	GGNN + Conv	8×BERT	LightGBM / TF-IDF+
Señal	+10.51 % acc., +8.68 % F1	MAE 0.73	cascada + compromiso

III. METODOLOGÍA

III-A. Definición del problema de aprendizaje

Sea x la representación de un evento de seguridad y $y \in \{\text{benign}, \text{suspicious}, \text{malicious}\}$ su nivel. El objetivo es aprender $f : x \mapsto (y, p)$, donde p es un vector de probabilidades por clase. La representación x se construye en tres niveles: campos tabulares del evento, el mensaje crudo saneado, y un subgrafo de su vecindario temporal.

Se trata de un problema de **clasificación supervisada multiclase** con tres características que condicionan el diseño: desbalance extremo (245:1), distribución no estacionaria en el tiempo, y presencia de un humano en el bucle de decisión. Por ello la tarea no se reduce a maximizar exactitud.

III-B. Conjunto de datos

Se emplea `witfoo/precinct6-cybersecurity` v2.1.0 (licencia Apache-2.0). La captura viva contiene

2,011,674 eventos con 38 columnas por señal. La distribución de clases se presenta en la tabla II. Adicionalmente se dispone de 238,511 señales históricas de incidentes, 47,585 nodos y 1,740,438 aristas de proveniencia, y 51,371 incidentes con reporte asociado.

Cuadro II
DISTRIBUCIÓN DE NIVELES EN LA CAPTURA VIVA.

Nivel	Eventos	Proporción
benign	1,897,153	94.31 %
suspicious	106,793	5.31 %
malicious	7,728	0.38 %

Las figuras 1 muestran la exploración inicial: el desbalance y la cobertura temporal. Se observa que los leads maliciosos terminan el 30 de julio mientras el tráfico benigno continúa hasta el 1 de agosto.

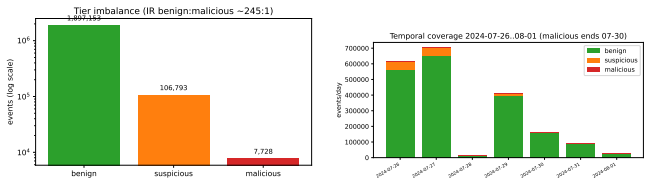


Figura 1. Exploración inicial: desbalance de clases (escala logarítmica) y cobertura temporal.

III-C. *Preprocesamiento*

Se especifican cinco metodologías, cumpliendo el requisito de al menos tres. **P1**: codificación categórica consciente del saneamiento, en la que los valores ausentes se conservan como un nivel propio en lugar de imputarse por tendencia central. **P2**: derivación de características temporales y partición temporal para evitar fugas de información. **P3**: manejo del desbalance por muestreo estratificado y ponderación de clases, descartando sobremuestreo sintético sobre variables codificadas. **P4**: vectorización del texto del registro. **P5**: construcción de subgrafos de vecindario en una ventana de ± 5 minutos compartiendo host o credencial.

III-D. *Modelos y análisis de complejidad*

Se proponen tres modelos. **M1**, un modelo tabular de gradient boosting con ponderación de clases, cuyo costo de entrenamiento crece con el número de árboles, las filas y las características binarizadas, y cuya inferencia es un recorrido raíz-hoja por árbol. **M2**, un modelo de texto que concatena la vectorización del mensaje con las características tabulares; su costo adicional es la vectorización, lineal en la longitud del mensaje. **M3**, un modelo de grafo que agrega el vecindario en la ventana temporal y alimenta un clasificador; su costo por evento depende del tamaño del vecindario. El orden de servicio previsto es una cascada: M1 ordena la cola, M2 confirma con evidencia textual y M3 confirma con evidencia estructural.

III-E. *Protocolo de evaluación*

La partición será **temporal**: se entrena con el pasado y se evalúa con el futuro, usando el cuantil 80 de la marca de tiempo como corte. Las métricas serán F1-macro y área bajo la curva precisión-recuerdo por clase, complementadas con el recall de la clase maliciosa a una tasa de falsos positivos fija, coherente con la asimetría de costos discutida. La exactitud se reportará únicamente para evidenciar que no es informativa bajo esta razón de desbalance.

IV. CONCLUSIÓN Y TRABAJO FUTURO

Este documento presentó el diseño de un estudio sobre triage de alertas SOC con desbalance extremo y deriva temporal. Se justificó el problema, se revisaron dos casos de literatura con resultados y limitaciones verificadas, y se especificó la metodología completa: representación, preprocesamiento, modelos y protocolo de evaluación.

Se anticipan las siguientes limitaciones. Primero, las etiquetas provienen de la correlación automática de la plataforma y no de un analista, por lo que constituyen un oráculo imperfecto; la mayoría de las señales permanecen sin procesar. Segundo, la distribución temporal y por organización es desigual, lo que restringe la generalización. Tercero, de forma análoga a lo documentado para CVSS [3]–[5], la etiqueta depende del asignador y por tanto del contexto.

El trabajo futuro contempla extender la tarea a predicción multi-etiqueta de técnicas ATT&CK y explorar codificadores contextuales entrenables sobre los reportes de incidente disponibles.

REFERENCIAS

[1] Y. Zhou, S. Liu, J. Siow, X. Du y Y. Liu, "Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks," en *NeurIPS*, 2019. arXiv:1909.03496.

[2] M. R. Shahid y H. Debar, "CVSS-BERT: explainable natural language processing to determine the severity of a computer security vulnerability from its description," en *IEEE ICMLA*, 2021, doi 10.1109/ICMLA52953.2021.00256. arXiv:2111.08510.

[3] S. Schwab y B. Gwerder, "An investigation into vulnerability databases," Universidad de Berna, 2020.

[4] "How context impacts vulnerability severity: an analysis of product-specific CVSS scores," en *LADC*, 2024, doi 10.1145/3697090.3697109.

[5] T. H. M. Le *et al.*, "An investigation into inconsistency of software vulnerability severity across data sources," arXiv:2112.10356, 2021.

[6] WitFoo, Inc., "WitFoo Precinct6 cybersecurity dataset v2.1.0," HuggingFace, 2026.

[7] MITRE, "ATT&CK design and philosophy," 2020. `attack.mitre.org`.

[8] D. Huang *et al.*, "NODLINK: online APT detection," en *NDSS*, 2024. arXiv:2311.02331.

[9] "KnowHow: CTI + provenance, interpretable," en *NDSS*, 2026. arXiv:2509.05698.

[10] "FLASH: GNN + semantic-temporal embedding," en *IEEE S&P*, 2024, doi 10.1109/SP54263.2024.00139.

[11] S. Wang *et al.*, "threaTrace: detecting host threats via GraphSAGE," arXiv:2111.04333, 2021.

[12] W. Hamilton, Z. Ying y J. Leskovec, "Inductive representation learning on large graphs," en *NeurIPS*, 2017.

[13] MITRE Center for Threat-Informed Defense, "Threat report ATT&CK mapper (TRAM)," `github.com/center-for-threat-informed-defense/tram`.

[14] R. Othman, B. Rossi y B. Russo, "A comparison of vulnerability feature extraction methods from textual attack patterns," arXiv:2407.06753, 2024.

[15] T.-Y. Lin, P. Goyal, R. Girshick, K. He y P. Dollár, “Focal loss for dense object detection,” en *ICCV*, 2017.
[16] “Cost-sensitive intrusion detection,” arXiv:0807.2043, 2008.